ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Кафедра «Компьютерной безопасности»

Отчет
По Заданию «А5»
По дисциплине «Язык Ассемблер»

Работу выполнил
Студент группы СКБ-242 _____ Флигинский В.М.

Работу проверила _____ Белова Е.В.

Москва, 2026

Начальные данные

Флигинский	Виктор Михайлович	vmfliginskii@edu.hse.ru	37

37. В массиве слов (байтов) выбранного размера (5-6 элементов) над каждым элементом выполнить операцию: если биты 6:5 являются конъюнкцией битовых полей 4:3 и 1:0, то инвертировать старший бит. Вывод на экран исходного массива и массива результатов производить в двоичном и шестнадцатеричном представлениях.

Текст программы

```
#include <stdio.h>

void printBin8(unsigned char val) {
   for (int i = 7; i >= 0; i--)
       printf("%d", (val >> i) & 1);
}

void printBin16(unsigned short val) {
   for (int i = 15; i >= 0; i--) {
       printf("%d", (val >> i) & 1);
       if (i == 8) printf(" ");
   }
}

void processByte(unsigned char *A, int n) {
   __asm {
       mov esi, A              ; адрес начала массива A в esi
       mov ecx, n              ; количество элементов в ecx (счётчик цикла)

   loop_byte:
       mov al, [esi]           ; текущий элемент массива в al
       mov bl, al              ; копия в bl для выделения поля 4:3
       mov dl, al              ; копия в dl для выделения поля 1:0

       and al, 01100000b       ; маска поля 6:5
       shr al, 5               ; прижать поле 6:5 к правому краю

       and bl, 00011000b       ; маска поля 4:3
       shr bl, 3               ; прижать поле 4:3 к правому краю

       and dl, 00000011b       ; маска поля 1:0 (уже прижата)

       and bl, dl              ; bl = (поле 4:3) AND (поле 1:0)

       cmp al, bl              ; сравнить поле 6:5 с конъюнкцией
       jne skip_byte           ; не совпало - пропустить инверсию

       xor byte ptr [esi], 10000000b  ; инвертировать старший бит
```

```
    skip_byte:
        inc esi                 ; перейти к следующему байту
        loop loop_byte          ; повторить цикл для следующего элемента

        nop                     ; конец ассемблерной вставки
    }
}

void processWord(unsigned short *A, int n) {
    __asm {
        mov esi, A              ; адрес начала массива A в esi
        mov ecx, n              ; количество элементов в ecx

    loop_word:
        mov ax, [esi]           ; текущее слово в ax
        mov bx, ax              ; копия в bx для поля 4:3
        mov dx, ax              ; копия в dx для поля 1:0

        and ax, 0000000001100000b  ; маска поля 6:5
        shr ax, 5                  ; прижать поле 6:5 к правому краю

        and bx, 0000000000011000b  ; маска поля 4:3
        shr bx, 3                  ; прижать поле 4:3 к правому краю

        and dx, 0000000000000011b  ; маска поля 1:0

        and bx, dx              ; bx = (поле 4:3) AND (поле 1:0)

        cmp ax, bx              ; сравнить поле 6:5 с конъюнкцией
        jne skip_word           ; не совпало — пропустить инверсию

        xor word ptr [esi], 1000000000000000b  ; инвертировать старший бит

    skip_word:
        add esi, 2              ; перейти к следующему слову (+2 байта)
        loop loop_word          ; повторить цикл для следующего элемента

        nop                     ; конец ассемблерной вставки
    }
}


void printArrayByte(unsigned char *A, int n) {
    for (int i = 0; i < n; i++) {
        printf("[%d] ", i);
        printBin8(A[i]);
        printf(" 0x%02X\n", A[i]);
```

```
    }
    printf("\n");
}

void printArrayWord(unsigned short *A, int n) {
    for (int i = 0; i < n; i++) {
        printf("[%d] ", i);
        printBin16(A[i]);
        printf(" 0x%04X\n", A[i]);
    }
    printf("\n");
}

int main() {

    unsigned char byteArr[6] = { 0x05, 0x7B, 0x65, 0x37, 0x1B, 0x6F };
    unsigned short wordArr[6] = { 0x0005, 0x007B, 0x1265, 0xAB37, 0x001B, 0x556F };

    printf("BYTES BEFORE:\n");
    printArrayByte(byteArr, 6);

    processByte(byteArr, 6);

    printf("BYTES AFTER:\n");
    printArrayByte(byteArr, 6);

    printf("WORDS BEFORE:\n");
    printArrayWord(wordArr, 6);

    processWord(wordArr, 6);

    printf("WORDS AFTER:\n");
    printArrayWord(wordArr, 6);

    return 0;
}
```

Тест программы

```
PS D:\HSE\Ассемблер\A2_victor> .\A5.exe
BYTES BEFORE:

[0] 00000101 0x05
[1] 01111011 0x7B
[2] 01100101 0x65
[3] 00110111 0x37
[4] 00011011 0x1B
[5] 01101111 0x6F

BYTES AFTER:

[0] 10000101 0x85
[1] 11111011 0xFB
[2] 01100101 0x65
[3] 00110111 0x37
[4] 00011011 0x1B
[5] 01101111 0x6F

WORDS BEFORE:

[0] 00000000 00000101 0x0005
[1] 00000000 01111011 0x007B
[2] 00010010 01100101 0x1265
[3] 10101011 00110111 0xAB37
[4] 00000000 00011011 0x001B
[5] 01010101 01101111 0x556F

WORDS AFTER:

[0] 10000000 00000101 0x8005
[1] 10000000 01111011 0x807B
[2] 00010010 01100101 0x1265
[3] 10101011 00110111 0xAB37
[4] 00000000 00011011 0x001B
[5] 01010101 01101111 0x556F
```